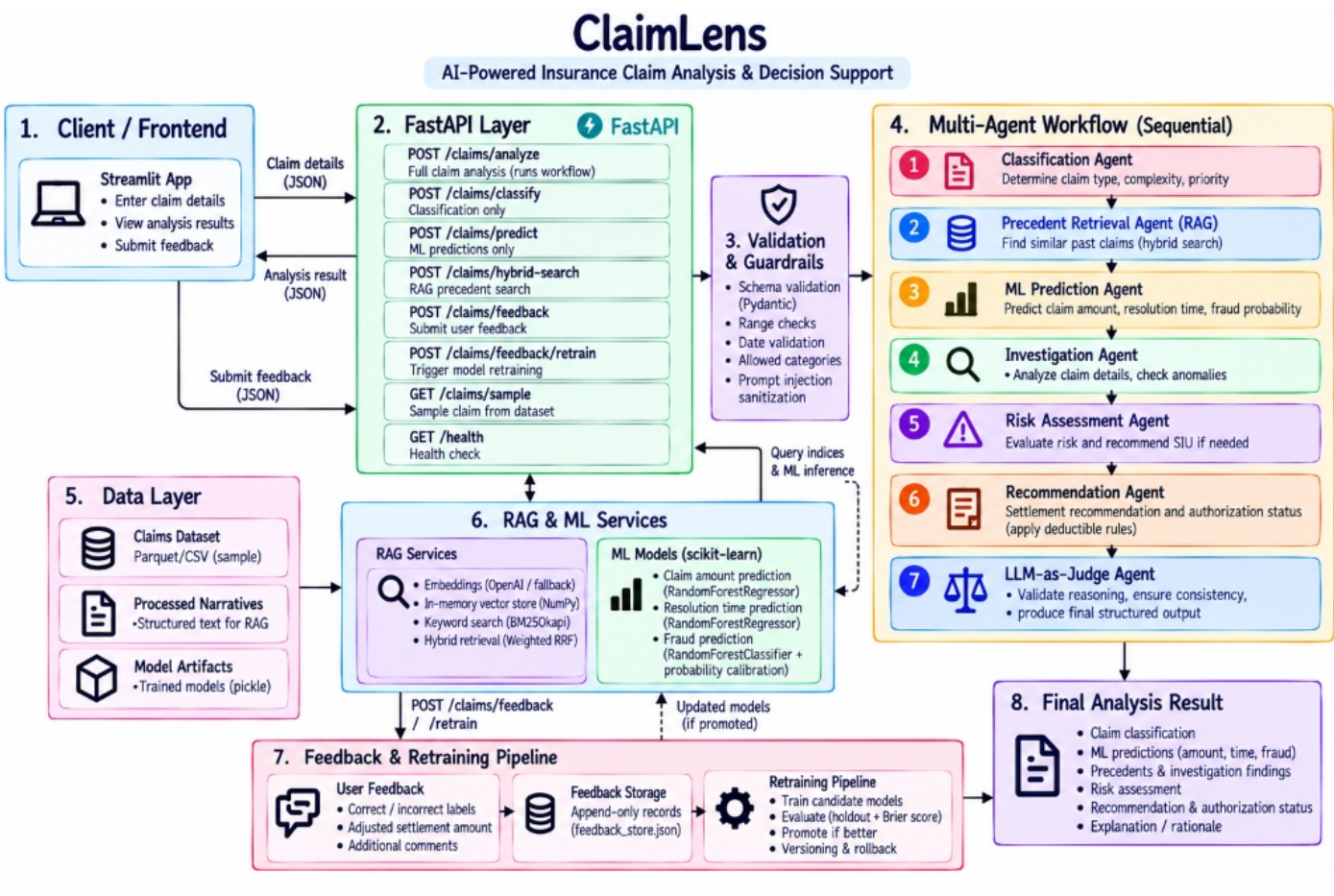


ClaimLens — System Architecture Diagram

End-to-End System Architecture: Client UI → FastAPI Microservice → Guardrails Gate → Multi-Agent Workflow Engine → Hybrid RAG & ML Services → Continuous Feedback Loop



#	Subsystem Component	Architectural Responsibility & Technologies
1	Client / Frontend	Streamlit Adjuster Dashboard (frontend.py) and External REST API clients.
2	FastAPI Layer	Asynchronous gateway (port 8000) exposing /analyze, /classify, /predict, /hybrid-search, /feedback, /feedback/retrain
3	Validation & Guardrails	Pydantic schema validation, range checks, date validation, and prompt injection sanitization.
4	Multi-Agent Workflow Engine	Coordinated reasoning: 1.Classification & Triage & 2.Investigation (RAG) & 3.Risk Assessment (ML) & 4.
5	Data Layer	Parquet/CSV samples (Hugging Face 100M claims), processed narrative texts, and versioned pickle model bundles.
6	RAG & ML Services	Hybrid RAG (NumPy in-memory vector store + BM25Okapi + Weighted RRF) and Calibrated Random Forests (loss,
7	Feedback & Retraining Pipeline	Append-only adjuster audit feedback store, automated holdout evaluation, Brier score gate, and zero-downtime hot r
8	Final Analysis Result	Standardized JSON payload returned to client containing classification, benchmarks, findings, invariants, and judge v

Architectural Invariants & Security: Deterministic financial settlement bounds (payout capped at $\max(0, \text{claim_amount} - \text{deductible})$), calibrated fraud risk distribution, and independent LLM-as-Judge audit pass rate of 9.3/10.